

Zadanie 02

Natalia Włódyka

Grudzień 7, 2025

1 Polecenie zadania

Celem zadania jest rozwiązanie równania $\mathbf{Ax} = \mathbf{e}$, gdzie $\mathbf{A}$ jest macierza podana poniżej, a $\mathbf{e}$ jest wektorem, którego wszystkie składowe są równe 1, za pomocą metody Gaussa-Seidela oraz gradientów sprzężonych.

2 Zadana macierz

Macierz $\mathbf{A}$ ma postać:

$$\mathbf{A} = \begin{bmatrix} 4 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 & \dots \\ 1 & 4 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 & \dots \\ 0 & 1 & 4 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & 0 & \dots \\ 0 & 0 & 1 & 4 & 1 & 0 & 0 & 1 & 0 & 0 & 0 & \dots \\ 1 & 0 & 0 & 1 & 4 & 1 & 0 & 0 & 1 & 0 & 0 & \dots \\ 0 & 1 & 0 & 0 & 1 & 4 & 1 & 0 & 0 & 1 & 0 & \dots \\ 0 & 0 & 1 & 0 & 0 & 1 & 4 & 1 & 0 & 0 & 1 & \dots \\ \dots & \dots & \dots & \dots & \dots & \dots & \dots & \dots & \dots & \dots & \dots & \dots \\ \dots & 0 & 0 & 1 & 0 & 0 & 1 & 4 & 1 & 0 & 0 & 1 \\ \dots & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 4 & 1 & 0 & 0 \\ \dots & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 4 & 1 & 0 \\ \dots & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 4 & 1 \\ \dots & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 1 & 4 \end{bmatrix}$$

Jak widać powyżej, jest to macierz pasmowa (inaczej wstęgowa). Wykorzystamy w metodzie calculateA, aby zmniejszyć koszt obliczeń do $O(n)$. Dla uproszczenia zadania elementy na przekątnej zapiszemy w wektorze $\mathbf{D}$, o długości równej 128.

3 Metoda Gaussa- Seidela

Macierz $\mathbf{A}$ posiada niezerowe elementy jedynie na przekątnej, oraz na diagonalach oddalonych o 1 i 4 po jej obu stronach. Z racji, że każdy element

znajdujący się po obu stronach przekątnej jest równy 1 lub 0, możemy wzór iteracyjny metody Gaussa-Seidela zapisać w postaci:

$$x_i^{k+1} = \frac{1}{4}(1 - x_{-1}^{k+1} - x_{i-4}^{k+1} - x_{i+1}^k - x_{i+4}^k)$$

4 Metoda gradientów sprzężonych

Metoda ta polega na iterawoniu zmiennych dopóki $\|r_k\| > \varepsilon$. Początkowe zmienne są podane poniżej:

$$x_0 = 0$$

$$r_0 = e - Ax_0$$

$$p_0 = r_0$$

Dla kolejnych $k = 0, 1, 2, 3 \dots$

$$\alpha_k = \frac{r_k^T r_k}{p_k^T A p_k}$$

$$r_{k+1} = r_k - \alpha_k A p_k$$

$$\beta_k = \frac{r_{k+1}^T r_{k+1}}{r_k^T r_k}$$

$$p_{k+1} = r_{k+1} + \beta_k p_k$$

$$x_{k+1} = x_k + \alpha_k p_k$$

5 Kod w Pythonie

```
import numpy as np
import matplotlib.pyplot as plt
from numba import njit
from numpy.typing import NDArray

vector = NDArray[np.float64]

def gauss_seidel_pom(x: vector, previous_x : vector, D: vector):
    n = len(x)
    x[0] = (1.0 - previous_x[1] - previous_x[4]) / D[0]

    for i in range(1, 4):
        x[i] = (1.0 - x[i - 1] - previous_x[i + 1] - previous_x[i + 4]) / D[i]

    for i in range(4, n - 4):
        x[i] = (1.0 - x[i - 1] - previous_x[i + 1] - x[i - 4] - previous_x[i + 4]
```

```

for i in range(n - 4, n - 1):
    x[i] = (1.0 - x[i - 1] - x[i - 4] - previous_x[i + 1]) / D[i]

x[n - 1] = (1.0 - x[n - 2] - x[n - 5]) / D[n - 1]

def gauss_seidel(D: vector) -> tuple[int, list[np.float64]]:
    number = 0
    n = len(D)
    x: vector = np.zeros(n, dtype=np.float64)
    previous_x: vector = np.zeros(n, dtype=np.float64)
    history_of_gauss = []

    for number in range(1, 101):
        previous_x[:] = x
        gauss_seidel_pom(x, previous_x, D)
        norm = np.linalg.norm(x - previous_x)
        history_of_gauss.append(norm)
        if norm < 1e-12:
            break

    return number, history_of_gauss

def conjugate_gradient(previous_x : vector, D: vector) -> tuple[int, list[np.float64]]:
    x = previous_x.copy()
    r = 1.0 - calculate_A(x, D)
    p = r.copy()
    previous_rTr = np.dot(r, r)

    history = [] # lista norm
    i = 1
    for i in range(1, 101):
        Ap = calculate_A(p, D)
        a = previous_rTr / np.dot(p, Ap)
        r = r - a * Ap
        new_x = x + a * p
        history.append(np.linalg.norm(new_x - x))

        if np.linalg.norm(r) < 1e-12:
            break

    new_rTr = np.dot(r, r)
    beta = new_rTr / previous_rTr
    p = r + beta * p

    x = new_x,

```

```

        previous_rTr = new_rTr

    return i + 1 , history

@njit(fastmath=True)
def calculate_A(x: vector , D: vector) -> vector:
    y = D * x
    n = len(x)

    for i in range(n - 4):
        y[i + 1] += x[i]
        y[i] += x[i + 1]

        y[i + 4] += x[i]
        y[i] += x[i + 4]

    for i in range(n - 4 , n - 1):
        y[i + 1] += x[i]
        y[i] += x[i + 1]

    return y

def main() -> None:
    #Metoda gaussa-seidela
    D: vector = np.full(128, 4.0)
    iterations , history_of_gauss = gauss_seidel(D)
    print(f"iterations: {iterations}")

    #Metoda gradientow sprzezonych
    x = np.zeros(128, dtype=np.float64)
    iterations_2 , history_of_gradients = conjugate_gradient(x, D)
    print(f"iterations_2: {iterations_2}")

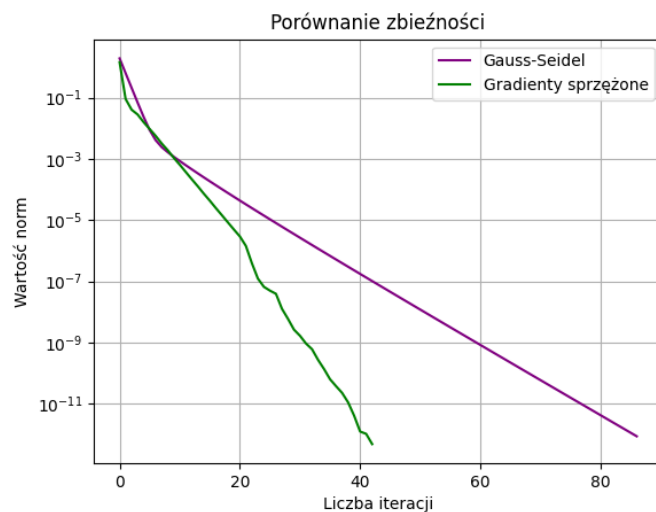
    #Tworzymy wykres
    plt.plot(history_of_gauss , label="Gauss-Seidel", color="purple")
    plt.plot(history_of_gradients , label="Gradienty sprzezone", color="green")
    plt.title("Porownanie zbieznosci")
    plt.xlabel("Liczba iteracji")
    plt.ylabel("Wartosc norm")
    plt.legend()
    plt.grid(True)
    plt.yscale("log")
    plt.show()

main()

```

6 Porównanie zbieżności norm

Jak widać na załączonym poniżej wykresie, liczba iteracji dla metody gradientów sprzężonych (ok. $40\mathbf{O(n)}$) jest mniejsza niż dla Gaussa-Seidela(ok. $80\mathbf{O(n)}$).



Dla rozkładu Choleskiego dla macierzy wstęgowej, dla której $k = 4$, koszt jest równy ok. $30\mathbf{O(n)}$, tym samym jest to metoda o największej efektywności wśród tych omówionych w zadaniu.